A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

MINI PROJECT REPORT

A Stock Market Data Analyzer in Python

Reading live NSE prices and reporting return, risk and trend from one Python file

Submitted by

Dax Varasadiya

Item	Detail
Enrolment No.	12402080601026
Division	IT-A
Subject	Python programming
Topic	Stock Market Data Analyzer
Concepts covered	File/API data, lists, dictionaries, GUI

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled “A Stock Market Data Analyzer in Python” was carried out by Dax Varasadiya (12402080601026) at A D Patel Institute of Technology in partial fulfilment of the requirements for the Bachelor of Technology degree in Information Technology. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Date: 26/10/26

Place: New Vallabh Vidyanagar

Lab Teacher

Prof. Nayan Mali
Assistant Professor

Head of Department

Dr Priyang Bhatt
Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the opportunity to undertake this mini project, “A Stock Market Data Analyzer in Python”, as part of the Python programming course.

I am thankful to my faculty guide for the guidance and suggestions that supported the planning and preparation of this project. The project provided an opportunity to apply Python to a practical problem and to understand how implementation choices affect performance and clarity.

I also appreciate the support of my classmates and everyone who offered assistance during the project. Finally, I am grateful to my family for their continued support.

Dax Varasadiya
(12402080601026)

TABLE OF CONTENTS

Certificate 2
Acknowledgement 3
List of Tables 5

1 Abstract 6
2 Introduction 6
3 Objectives 6
4 Concepts and technologies used 7
5 System requirements 7
6 System design 8
7 Implementation 9
8 Data source 10
9 Analysis methods 10
10 User interface 11
11 Testing and results 11
12 Limitations 12
13 Future scope 13
14 Conclusion 13
15 References 13
Appendix A How to run the program 14

LIST OF TABLES

Table 1 Concepts and technologies used 7
Table 2 System requirements 7
Table 3 Functions grouped by pipeline stage 8
Table 4 Testing and results 11
Table 5 Command reference 14

1 Abstract

This report describes a stock market data analyzer written in Python. The program reads live price history for Indian shares from a public Yahoo Finance endpoint that needs no API key and no registration, and turns that history into a small set of statistics a reader can act on: total return, annualised volatility, maximum drawdown, two simple moving averages and a trend signal.

The whole tool is a single file of 852 lines that uses only the Python standard library. It offers a Tkinter desktop window, a command-line interface and an offline demonstration mode, and it can save any series to a CSV file for later analysis without a network connection. Every concept the project list assigns to this topic — API and file data, lists, dictionaries and a graphical interface — appears in the finished program, and the report names the function where each one lives.

The program was tested against the live endpoint on 26 October 2026. It returned 247 daily bars for RELIANCE.NS, 75 five-minute bars for a single session, handled an invalid symbol without crashing, and produced a full report with no network connection in demonstration mode. One defect was found and is recorded in section 11.

2 Introduction

Price data for Indian shares is easy to look at and awkward to work with. A student who wants to measure how a share actually behaved over a year has three options, and all three are inconvenient. The first is a broker's app, which shows a chart but hides the underlying numbers and offers no way to export them. The second is a paid data provider, which requires a subscription and an API key. The third is a spreadsheet, which means typing prices by hand and is only as reliable as the typing.

This project takes a fourth route. Yahoo Finance publishes a chart endpoint that answers without registration and returns daily and intraday bars for Indian shares listed on the National Stock Exchange and the Bombay Stock Exchange. The program calls that endpoint, parses the response, and computes the measures that describe a price series. The result is a tool a student can run on an ordinary laptop, with no account and no cost, that produces numbers rather than a picture.

The project was chosen from the supplied topic list at number 75, whose assigned concepts are file and API data, lists, dictionaries and a graphical user interface. The implementation keeps to that brief and deliberately avoids a database, so that the project can be demonstrated on a machine where installing extra software may be restricted.

3 Objectives

The program was built to satisfy four objectives, in this order of priority.

1. Read any Indian share. The user should be able to type a ticker directly, or search by company name, and reach any share listed on the NSE or the BSE.
2. Report the numbers that matter. For a chosen period the program should give the total return, the annualised volatility, the maximum drawdown, the highest and lowest price with their dates, and a simple trend signal.

3. Work at any timeframe. The same analysis should run on one-minute candles, on five-minute and fifteen-minute candles, on hourly bars, and on daily and weekly data.
4. Run without setup. The program should need no API key, no database and no third-party package, and should still demonstrate itself when there is no internet connection.

4 Concepts and technologies used

The project list assigns four concepts to this topic. Each one is used in a specific place, and the table below names that place so the concept can be pointed to during a demonstration.

Table 1: Concepts and technologies used

Concept	How it is used	Where in the program
Lists	Dates, closing prices and volumes are held as three parallel lists; moving averages and daily returns are also lists.	fetch_stock, simple_moving_average, daily_returns
Dictionaries	Every computed statistic is returned as one labelled dictionary, and the search results are a list of dictionaries.	analyse, market_overview, search_symbols
File handling	A price series is written to a CSV file and read back with the csv module.	save_csv, load_csv
GUI	A Tkinter window carries the form, the results table, the chart and the indices box.	launch_gui

Three further language features appear throughout, although the topic list does not require them: functions, since the program is built from nineteen of them; exception handling, which wraps every network and file operation; and modules, since the program imports csv, json, math, statistics, urllib and datetime from the standard library.

5 System requirements

Table 2: System requirements

Item	Requirement
Hardware	Any modern x86 or ARM processor; 4 GB of RAM; under 10 MB of disk
Operating system	Windows, macOS or Linux, wherever Python runs
Python	Version 3.8 or later
Libraries	Standard library only; matplotlib is optional and only adds the PNG chart

The program is a single file, so installation consists of copying that file to a folder. There is no installer, no configuration file and no environment variable to set. Live data needs an internet connection; demonstration mode does not, which is why the program can be shown on a machine with no network.

6 System design

The program is organised as a pipeline of four stages. A symbol enters, a price series is downloaded, the series is analysed, and the result is drawn. Only the third stage performs arithmetic; the other three move data or display it.

1. Symbol. The user types a ticker, picks one from the drop-down, or searches a company name and chooses from the results.
2. Download. fetch_stock builds a URL from the symbol, the range and the interval, calls the endpoint, and parses the JSON response into three parallel lists.
3. Analyse. analyse takes those lists and returns one dictionary containing every statistic the program reports.
4. Render. format_report turns the dictionary into text, build_chart or ascii_chart draws the price series, and launch_gui places both in the window.

The nineteen functions divide cleanly across those stages, as the table below shows.

Table 3: Functions grouped by pipeline stage

Stage	Functions	Purpose
Input	search_symbols, is_indian	Find a share by name and keep only Indian listings
Download	fetch_stock, market_overview	Call the endpoint and parse the response
Analyse	simple_moving_average, daily_returns, max_drawdown, analyse, periods_per_year, is_intraday	Compute the statistics
Render	format_report, build_chart, ascii_chart, mpl_available	Turn numbers into text and a chart
Interface	launch_gui, main	Desktop window and command line
Storage	save_csv, load_csv	Write and read a CSV file
Offline	make_demo_data	Generate sample data when there is no network

7 Implementation

7.1 Data acquisition

fetch_stock is the only function that touches the network. It builds the request URL from three pieces: the symbol, a range such as 1y, and an interval such as 1d or 5m. It sends the request with a browser user-agent, decodes the JSON, and reads the timestamp array together with the closing prices and volumes from the quote object. Rows whose closing price is null, which is how the endpoint marks a holiday, are skipped.

Two details matter here. First, intraday timestamps are shifted by the exchange's own UTC offset before they are formatted, so a five-minute series for the NSE reads 09:15 to 15:30 regardless of the timezone the computer is set to. Second, every failure is converted into a plain message rather than an exception, so a misspelt symbol produces a sentence and not a traceback.

7.2 Search by company name

search_symbols calls a second public endpoint that takes a free-text query and returns matching instruments. The response contains listings from many exchanges, so the results pass through is_indian, which keeps only symbols ending in .NS for the NSE, .BO for the BSE, or beginning with a caret for an index. The search is therefore a way to reach any Indian share without knowing its ticker, and the hard-coded drop-down becomes a shortcut rather than a limit.

7.3 Analysis

simple_moving_average computes a running average over a window and returns a list the same length as the input, with None in the positions where there are not yet enough values. daily_returns turns a price list into a list of percentage changes. max_drawdown walks the series once, tracking the highest price seen so far, and records the deepest fall from that peak. analyse calls all three, adds the high, the low, the average, the best and worst bar, and returns the whole set as one dictionary.

7.4 Report and chart

format_report lays the dictionary out as an aligned text report with a heading, a period line and four labelled groups. The labels adapt to the timeframe: a daily series reports trading days and best and worst days, while an intraday series reports bars and best and worst moves. build_chart draws the closing price with the two moving averages using matplotlib, and spaces the time axis according to the interval, so a five-minute series is labelled in minutes rather than hours. When matplotlib is not installed, ascii_chart draws the same series with characters instead.

7.5 Interface

launch_gui builds a Tkinter window with four regions: a control bar holding the symbol, the timeframe and the two action buttons; a search bar with a results list; a market indices box showing the Nifty 50, the Sensex and the Bank Nifty; and a body holding the text report beside the chart. The window never blocks on a failure; every error appears as a dialog and the application stays open.

7.6 Offline demonstration

make_demo_data generates a plausible price series with a random walk and a fixed seed, so the program can be demonstrated on a machine with no internet connection. The same analysis, report and chart run on that data, which means the demonstration does not depend on the network being available in the examination room.

8 Data source

All live data comes from the public chart endpoint at query1.finance.yahoo.com, which returns a JSON document containing a timestamp array and arrays of open, high, low, close and volume. No key, token or registration is required. The search feature uses the companion search endpoint on the same host.

Indian shares are identified by suffix: .NS for the National Stock Exchange and .BO for the Bombay Stock Exchange. Indices begin with a caret, as in ^NSEI for the Nifty 50 and ^BSESN for the Sensex. The program filters search results to those forms, so a query such as infosys returns INFY.NS and INFY.BO but not the New York listing.

Because the endpoint is public and undocumented, it is treated as the one fragile part of the design and isolated inside a single function. If the endpoint changes, one function changes; the analysis, the report and the interface are unaffected.

9 Analysis methods

The program computes five measures. Each is defined below in the form it is implemented.

Daily return. For each pair of consecutive prices, the percentage change is (P(t) − P(t−1)) / P(t−1) × 100. The list of these changes is the input to volatility and to the best and worst bar figures.

Simple moving average. For a window of n bars, SMA(n) at position t is the mean of the n closing prices ending at t. The program computes a 7-bar and a 30-bar average.

Annualised volatility. The standard deviation of the bar returns is multiplied by the square root of the number of bars in a trading year. That count depends on the interval: 252 for daily bars, 252 × 75 for five-minute bars, 52 for weekly bars. Scaling by the interval is what makes the figure comparable across timeframes.

Maximum drawdown. Walking the series once, the program tracks the running peak and records the largest percentage fall from that peak. The result is always zero or negative; −27.2 per cent for Reliance over the year to October 2026 means the share at its worst was 27.2 per cent below its own earlier high.

Trend signal. If the latest close is above the 30-bar average the program reports an uptrend, and otherwise a downtrend. This is a description of the present state, not a forecast, and the report says so.

10 User interface

The window opens at 1000 by 720 pixels. The control bar carries a symbol drop-down of seventeen Indian shares and indices, a timeframe drop-down of nine presets ranging from one-day one-minute candles to five-year weekly bars, and buttons for Analyze and Save chart. Below it, a search field accepts a company name and lists the matching Indian listings; double-clicking a row loads that symbol and runs the analysis immediately.

The market indices box shows the Nifty 50, the Sensex and the Bank Nifty side by side, each with its latest value and its change for the day, coloured according to direction, with a Refresh button. The body of the window holds the text report on the left and the chart on the right. When matplotlib is absent the chart

area stays empty and a text chart is appended to the report instead, so the window remains usable on a machine where the package cannot be installed.

11 Testing and results

Five checks were run against the live endpoint on 26 October 2026. The table records what each check did and what was observed.

Table 4: Testing and results

Check	Result	Observation
Live daily fetch	Pass	RELIANCE.NS returned 247 daily bars for a one-year range, from ₹1,484.10 to ₹1,198.00.
Intraday fetch	Pass	A one-day five-minute request returned 75 bars, labelled 09:15 to 15:25 in exchange time.
Invalid symbol	Pass	A misspelt ticker produced a clear message and no traceback.
Offline demonstration	Pass	The full report and a text chart were produced with no network connection.
CSV round trip	Partial	Prices reload correctly, but the interval is not stored, so a reloaded file is analysed as daily data.

The CSV result is the one defect the testing found. save_csv writes the date, the close and the volume, but not the interval that produced them. When the file is read back, the program assumes daily bars, which is wrong for an intraday series and makes the volatility figure too high. The fix is small: add the interval as a column or a header line and read it in load_csv.

The two figures below are real output from the program. The first is the daily chart for Reliance over one year, with the 7-day and 30-day averages drawn over the closing price. The second is a single session at five-minute resolution, where the time axis is labelled in minutes and the moving averages are computed on bars rather than days.

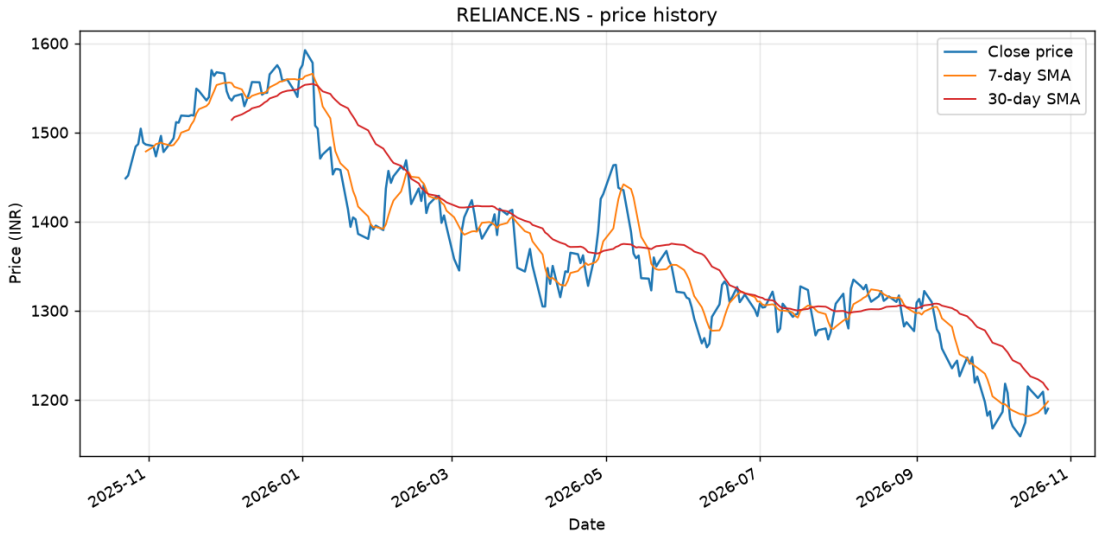


Figure 1 — RELIANCE.NS, one year of daily closes with the 7-day and 30-day moving averages

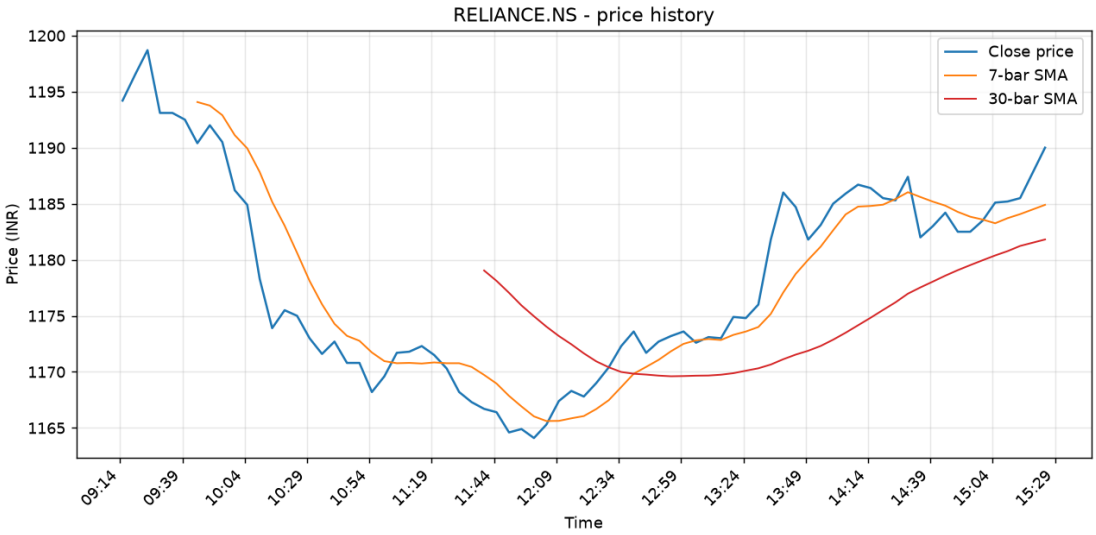


Figure 2 — RELIANCE.NS, one session at five-minute resolution

The summary figures for the year, as printed by the program, were a total return of −19.3 per cent, an annualised volatility of 21.2 per cent, a maximum drawdown of −27.2 per cent, a high of ₹1,592.30 on 2 January 2026 and a low of ₹1,158.90 on 12 October 2026.

12 Limitations

Four limits are worth stating plainly rather than leaving for a question.

- **Network dependence.** Live mode needs an internet connection. Without one, only the generated demonstration data is available.
- **One data source.** Everything rests on a single public endpoint that Yahoo could change or restrict without notice.
- **Simple indicators.** The program computes two moving averages, volatility and drawdown. It does not compute RSI, MACD or Bollinger bands, and it does not backtest a strategy.
- **No database.** Prices are saved to CSV files. There is no history of past runs, no watchlist and no SQL, which is a deliberate choice to keep the demonstration free of setup.

13 Future scope

The clearest next step is a database. A MySQL table holding a watchlist of shares and a second table holding saved analyses would turn the tool from a one-off calculator into something that accumulates. That step needs a server, a user account and the mysql-connector package, which is why the current version avoids it: on a locked-down machine, installing that package may not be possible.

Beyond the database, three additions would each be a self-contained piece of work: more indicators, with RSI and a moving-average crossover signal; a screener that runs the same analysis across a list of shares and ranks them by return or volatility; and an export to PDF so that a report can be printed without a screenshot.

14 Conclusion

The project set out to make Indian share prices analysable without a subscription, a key or a database, and it does that. One file of 852 lines reads live NSE and BSE prices, computes return, volatility, drawdown and trend at any timeframe from one minute to five years, and presents the result as a report, a chart and a desktop window. It covers the four concepts the topic list assigns to it, and it demonstrates itself offline when the network is unavailable.

The weaknesses are known and written down: a single data source, a small set of indicators, and a CSV format that loses the interval. Each has a defined fix. What the project demonstrates is that a useful piece of analysis does not require a large stack, and that a public endpoint, the standard library and a clear separation between fetching, analysing and drawing are enough.

15 References

1. Yahoo Finance chart endpoint, query1.finance.yahoo.com/v8/finance/chart. Accessed 26 October 2026.
2. Yahoo Finance search endpoint, query1.finance.yahoo.com/v1/finance/search. Accessed 26 October 2026.
3. Python Software Foundation, The Python Standard Library: csv, json, statistics, urllib, datetime. docs.python.org/3/library.
4. Python Software Foundation, tkinter — Python interface to Tcl/Tk. docs.python.org/3/library/tkinter.html.
5. Hunter, J. D., Matplotlib: A 2D graphics environment, Computing in Science and Engineering, 9(3), 2007.
6. National Stock Exchange of India, equity symbol conventions and trading hours, nseindia.com.

Appendix A How to run the program

Save stock_analyzer.py to a folder and open a terminal in that folder. The program needs Python 3.8 or later. matplotlib is optional and only affects whether the chart is a picture or text.

Table 5: Command reference

Command	What it does
python stock_analyzer.py	Opens the desktop window
python stock_analyzer.py --demo	Runs entirely offline with generated data
python stock_analyzer.py --symbol TCS.NS	Analyses one share in the terminal
python stock_analyzer.py --symbol TCS.NS --range 1d --interval 5m	Analyses one session at five-minute resolution
python stock_analyzer.py --indices	Prints the Nifty 50, Sensex and Bank Nifty
python stock_analyzer.py --search "tata motors"	Finds Indian shares by company name
python stock_analyzer.py --find "infosys"	Searches a name and analyses the top match
python stock_analyzer.py --symbol TCS.NS --chart tcs.png	Saves the chart as a PNG file
python stock_analyzer.py --csv prices.csv	Reads a previously saved price file

If the command python is not recognised, use python3. If matplotlib cannot be installed because of a security policy, the program still runs and prints a text chart in place of the picture; no change to the code is needed.